

ADVANCED COMPUTER ARCHITECTURE LAB PROJECT

<https://github.com/KondetiAravind/Cache-Prefetching-Schemes>

CACHE PREFETCHING SCHEMES & MEMORY MODELING USING SIMPLESCALAR + CACTI

Kondeti Aravind (22CS02008)

Gunupuru Sai Siddhartha (22CS02007)

Overview:

This project is executed in **three major phases**, each building upon the previous:

Phase 1 — SimpleScalar Setup & Baseline Simulation

Phase 2 — Cache Prefetching Schemes (Core of the Project)

- Modify cache.c and cache.h to integrate:
 - Next-Line Prefetching
 - One-Block Lookahead Prefetching
 - Stride Prefetching
- Execute full set of **12 benchmark runs**
- Extract IPC, CPI, miss rates, prefetch counts

Phase 3 — Memory Modeling + Power/Energy Analysis

- Parse SimpleScalar stats → JSON
- Compute:
 - AMAT
 - Total Energy
 - CPI/IPC improvements
- Integrate **CACTI energy results**
- Produce final combined table + plots

All three phases together provide a **complete performance + energy analysis pipeline**, moving from simulation → modeling → hardware-level cache energy estimation.

STEP 1 — Objective & Workflow

1.1 Objective

The main objective of this project is to:

Enhance cache performance in the SimpleScalar out-of-order simulator by adding hardware prefetching schemes and evaluate their effects on IPC, CPI, miss rates, cycle count and energy consumption.

This includes:

- Integrating **IL1 and DL1 cache prefetching** into SimpleScalar's **sim-outorder**
- Implementing and comparing:
 - **Baseline (no prefetching)**
 - **Next-Line Prefetching**
 - **One-Block Lookahead (OBL) Prefetching**
- Running four benchmarks under all modes
- Extending the study with:
 - **Memory AMAT modeling**
 - **Energy modeling using CACTI**
 - **Unified results and comparative analysis**

STEP 2 — Theoretical Background

2.1 Cache Hierarchy Review

Modern processors rely heavily on hierarchical caches to minimize average memory access time. Key parameters include:

- Hit latency
- Miss rate
- Replacement policy
- Line/block size
- Associativity

2.2 Memory Latency and AMAT

Average Memory Access Time (AMAT):

$$\text{AMAT} = T_{\text{hit}} + \text{MR} * T_{\text{miss}}$$

Where miss penalty includes:

- L2 hit/miss
- Main memory latency

2.3 Prefetching Motivation

Cache misses—particularly compulsory and capacity misses—stall pipelines. Prefetching attempts to bring blocks before demand.

Benefits

- Hides memory latency
- Reduces pipeline stalls
- Improves IPC

Risks

- Useless prefetches
- Pollution
- Increased bandwidth pressure

2.4 Prefetching Schemes Implemented

Scheme	Trigger	Description	Target	Primary Benefit
Baseline (No Prefetch)	—	Normal cache operation — no speculative fetches.	IL1, DL1	Serves as reference for all comparisons.
Next-Line Prefetch (Prefetch-on-Access)	On every access	Prefetches the immediate next cache line (address + block_size) upon each access (hit or miss).	IL1, DL1	Improves spatial locality for sequential instruction/data access.
One-Block Lookahead (OBL)	On each miss	Prefetches one additional block ahead (address + 2 × block_size), anticipating continuous sequential accesses.	IL1	Higher prefetch distance improves instruction streaming efficiency.

STEP 3 — Implementation Details

Modified source files:

~/Cache_Prefetching_Schemes/cache.c

~/Cache_Prefetching_Schemes/cache.h

- Added Fields in `cache.h`

```
unsigned long long prefetch_count; // count of total prefetches issued
int in_prefetch;                // flag to prevent recursive calls
```

- Initialization in `cache_create()`

```
cp->prefetch_count = 0;
cp->in_prefetch = 0;
```

- Prefetching Logic in `cache_access()`

```
if (!cp->in_prefetch) {
    const char *mode = getenv("PREFETCH_MODE");
    md_addr_t next_line = ((addr / cp->bsize) + 1) * cp->bsize;

    if (mode && strcmp(mode, "nextline") == 0) {
        cp->in_prefetch = 1;
        cache_access(cp, Read, next_line, NULL, cp->bsize, now, NULL,
NULL);
        cp->prefetch_count++;
        cp->in_prefetch = 0;
    }

    else if (mode && strcmp(mode, "oneblock") == 0) {
        md_addr_t next_block = ((addr / cp->bsize) + 2) * cp->bsize;
        cp->in_prefetch = 1;
        cache_access(cp, Read, next_block, NULL, cp->bsize, now, NULL,
NULL);
        cp->prefetch_count++;
        cp->in_prefetch = 0;
    }
}
```

```

else if (mode && strcmp(mode, "stride") == 0) {
    static md_addr_t last_addr = 0, stride = 0;
    md_addr_t new_stride = addr - last_addr;
    if (stride == new_stride) {
        cp->in_prefetch = 1;
        cache_access(cp, Read, addr + stride, NULL, cp->bsize, now,
NULL, NULL);
        cp->prefetch_count++;
        cp->in_prefetch = 0;
    }
    stride = new_stride;
    last_addr = addr;
}
}

```

- Statistics Registration

```

stat_reg_counter(sdb, "prefetches_issued",
    "Total number of prefetches issued", &cp->prefetch_count, 0, NULL);

```

STEP 4 — Experimental Setup and Environment

System

- Ubuntu Linux
- SimpleScalar PISA toolchain
- Python 3 for modeling scripts
- CACTI 6.x for cache energy estimation

Benchmarks Used

- test-math
- test-fmath
- anagram
- test-llong

STEP 5 — Execution Commands

Refer to commands.txt

[./Cache_Prefetching_Schemes/commands.txt](#)

STEP 6 — Simulation Results

Simulation Results (test-math)

Metric	Baseline	Next-Line	One-Block Lookahead
IPC (Inst/Cycle)	0.9354	1.2623	1.2777
CPI (Cycle/Inst)	1.0489	0.7922	0.7827
IL1 Prefetches	0	252770	252968
DL1 Prefetches	0	57052	57019
sim_cycle	224609	169635	167603

Simulation Results (test-fmath)

Metric	Baseline	Next-Line	One-Block Lookahead
IPC (Inst/Cycle)	0.7853	1.1303	1.1880
CPI (Cycle/Inst)	1.2734	0.8847	0.8417
IL1 Prefetches	0	63094	63376
DL1 Prefetches	0	16502	16527
sim_cycle	68635	47685	45366

Simulation Results (test-llong)

Metric	Baseline	Next-Line	One-Block Lookahead
IPC (Inst/Cycle)	0.9581	1.2736	1.3356
CPI (Cycle/Inst)	1.0437	0.7852	0.7487
IL1 Prefetches	0	33325	33415
DL1 Prefetches	0	10300	10302
sim_cycle	31395	23619	22521

Simulation Results (anagram)

Metric	Baseline	Next-Line	One-Block Lookahead
IPC (Inst/Cycle)	0.5680	0.9721	1.1057
CPI (Cycle/Inst)	1.7605	1.0287	0.9044
IL1 Prefetches	0	9729	9790
DL1 Prefetches	0	4246	4246
sim_cycle	14440	8437	7418

STEP 7 — Memory Modeling & AMAT Analysis

Using:

```
./Cache_Prefetching_Schemes/Memory_Modeling/scripts/parse_sim_stats.  
py  
./Cache_Prefetching_Schemes/Memory_Modeling/scripts/compute_amat_  
and_energy.py
```

We extracted:

- IPC
- CPI
- IL1/DL1/L2 AMAT (ns and cycles)
- Dynamic main-memory energy
- Combined system-level energy

This enables comparison across all modes and benchmarks.

STEP 8 — CACTI Energy Modeling

We cloned CACTI, created config files, and generated energy results for:

- IL1
- DL1
- L2

Extracted:

- Total dynamic read energy
- Total dynamic write energy
- Leakage power

These were merged with SimpleScalar-derived statistics into:

[results/summary_with_energy_final.csv](#)

STEP 9 — Analysis & Observations

9.1 Performance Improvement

Across all benchmarks:

- IPC improved by **30–40%**
- CPI reduced significantly
- Simulated cycles reduced by **20–30%**
- Prefetch count shows healthy usage without flooding memory

9.2 One-Block Lookahead Performs Best

- Highest IPC in all workloads
- Best latency hiding
- Prefetch distance of 2 blocks is optimal

9.3 Next-Line Prefetching

- Simple
- Reliable gains
- Best for sequential workloads (test-math, test-llong)

9.4 Workload Behavior

- **test-math/test-llong**: Very sequential → huge benefit
- **test-fmath**: Mixed behavior → moderate improvement
- **anagram**: Largest relative improvement (nearly 2× IPC)

9.5 Energy Analysis (CACTI)

Even though prefetching increases accesses:

- Total energy impact remains small (~micro-joules)
- IPC improvement gives **better performance per watt**
- Dynamic IL1/DL1 energy dominated results (L2 minimal)

9.6 AMAT

OBL & Next-Line drastically reduce L1 miss rates →
lower L2 accesses →
lower DRAM accesses →
lower AMAT.

STEP 10 — Conclusions

- **Hardware prefetching significantly enhances pipeline throughput.**
- **One-Block Lookahead is the best-performing strategy**, consistently improving IPC while keeping energy overhead low.
- The extended modeling with **AMAT + CACTI** provides a realistic hardware-level evaluation.
- The integration of simulation + modeling replicates a full academic microarchitecture study.

Overall: Prefetching provides strong gains in performance, reduces memory stalls, and demonstrates practical microarchitectural improvement.

Final Directory Structure

Cache_Prefetching_Schemes/

